In [ ]:
```
# ── COLAB SETUP — run this cell first ──────────────────────────────
# Installs the stack this notebook needs. Safe to re-run.
# Colab already ships numpy / scipy / matplotlib / requests, so only the
# Qiskit packages are installed here (upgrading the rest causes conflicts).
%pip install -q --upgrade "qiskit⩾1.2" "qiskit-aer⩾0.15"

import importlib, sys
for mod in ("qiskit", "qiskit_aer", "numpy", "scipy", "matplotlib",
"requests"):
    try:
        m = importlib.import_module(mod)
        print(f"  {mod:<12} {getattr(m, '__version__', 'ok')}")
    except Exception as exc:
        print(f"  {mod:<12} MISSING → {exc}")

print("\nSetup check done.")
print("If a package above is MISSING, use Runtime → Restart session, then
Run all.")
```

Challenge 1: Scalable GHZ Entanglement & Hardware Trade-Offs

Track: Beginner
Challenge ID: `challenge_beginner` (or `challenge_example_1`)
Theme: Unbounded Scoring, Circuit Depth, Routing Overheads, and Noise

---

Welcome to Challenge 1!

In introductory quantum computing, we learn how to create a 2-qubit Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

While a 2-qubit Bell state is foundational, modern quantum algorithms rely on entangling **large registers of qubits**. The N-qubit generalization is the **Greenberger–Horne–Zeilinger (GHZ) state**:

$$|GHZ_N\rangle = \frac{|0\rangle^{\otimes N} + |1\rangle^{\otimes N}}{\sqrt{2}} = \frac{|00\ldots0\rangle + |11\ldots1\rangle}{\sqrt{2}}$$

In this challenge, you will design, transpile, and optimize an N-qubit GHZ circuit. Rather than capping your performance at a fixed 1,000-point score, this challenge features an **unbounded ("infinite") scoring system**: the more qubits you entangle, the higher your potential score — but only if you successfully protect against **errors**, **SWAP gates**, and **circuit depth**!

How Challenges Are Graded: The Unbounded Scoring System

The grading platform evaluates your submission across four orthogonal dimensions:

$$\text{Final Score} = \text{round}\Big(\text{Base}(N) \times \mathcal{F}_{\text{error}} \times \mathcal{S}_{\text{swap}} \times \mathcal{D}_{\text{depth}}\Big)$$

1. Balanced Polynomial Base Scaling ($1,000 + 50 \times (N^2 - 4)$)

$$\text{Base}(N) = 1,000 + 50 \times (N^2 - 4)$$

- $N = 2$: 1,000 points (the classic Bell state baseline)
- $N = 4$: 1,600 points
- $N = 16$: 13,600 points
- $N = 30$: 45,800 points ($N = 156$: ~1,200,000 points) This strictly rewards scaling to higher qubit counts so that $N = 30$ is noticeably better than $N = 16$, while balancing sensitivity so that a high-fidelity, zero-SWAP $N = 28$ circuit beats an unoptimized $N = 30$ circuit.

2. Error Penalty (Punish Non-Correlated Outcomes)

For an N-qubit GHZ state, the only desired measurement outcomes are all-zeros ($00\ldots0$) and all-ones ($11\ldots1$). The correlation rate is:

$$C = \frac{\text{counts}(00\ldots0) + \text{counts}(11\ldots1)}{\sum \text{counts}}, \quad \epsilon = 1 - C$$

Under pure random noise across N qubits, the chance of measuring $0\ldots0$ or $1\ldots1$ by luck is $C_{\text{rand}} = \frac{2}{2^N} = \frac{1}{2^{N-1}}$.

- If $C \leq C_{\text{rand}}$, the state is indistinguishable from noise: **Score = 0**.
- Above random noise, fidelity factor is:

$$\mathcal{F}_{\text{error}} = \left(\frac{C - C_{\text{rand}}}{1 - C_{\text{rand}}}\right)^{1.5}$$

3. SWAP Penalty (Punish Routing Overhead)

Real quantum computers have sparse physical qubit connectivity (e.g., heavy-hex or linear topologies). If two qubits that need to interact are not directly connected, the transpiler must insert **SWAP gates**. Each SWAP gate requires **3 CNOT gates**, adding gate error and decoherence time.

$$\mathcal{S}_{\text{swap}} = \frac{1}{1 + 0.25 \times N_{\text{swap}}}$$

4. Depth Efficiency (Punish Idle & Unparallelized Gates)

The theoretical minimum gate depth for preparing an N-qubit GHZ state is:

$$D_{\text{min}} = 1\ (\text{H gate}) + \lceil \log_2 N \rceil\ (\text{parallel CNOT tree})$$

- A naive linear cascade ($0 \to 1 \to 2 \to 3\ldots$) has depth $O(N)$.
- A parallel binary tree achieves logarithmic depth $O(\log_2 N)$.

$$\mathcal{D}_{\text{depth}} = \frac{D_{\text{min}}}{\max(D_{\text{actual}}, D_{\text{min}})}$$

The 5 Pillars to Maximize Your Challenge Score

1. **Scale Qubits ($N \uparrow$):** Base potential scales quadratically ($1,000 + 50(N^2 - 4)$). Higher qubit counts unlock much higher scoring tiers.
2. **Error Mitigation:** Apply readout error mitigation (e.g. M3 or twirled readout) and dynamical decoupling to recover physical correlation (C) under noise.
3. **Circuit Depth Optimization ($D \to D_{\min}$):** Replace sequential linear CX ladders with parallel binary trees to keep critical path depth at $1 + \lceil \log_2 N \rceil$.
4. **Eliminate SWAP Gates ($N_{\text{swap}} = 0$):** Route two-qubit gates only between directly connected physical qubits to avoid the $1/(1 + 0.25 \times N_{\text{swap}})$ penalty.
5. **Hardware-Aware Layout:** Use `VF2Layout` or explicit `initial_layout` matching the native hardware coupling map.

In [ ]:
```
# Run once if dependencies are needed
# %pip install -q qiskit qiskit-aer matplotlib requests

import os
import math
import requests
import matplotlib.pyplot as plt

from qiskit import QuantumCircuit, transpile, qasm2
from qiskit.visualization import plot_histogram
from qiskit.transpiler import CouplingMap
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

print("Qiskit and dependencies loaded successfully.")
```

Part 1: The 2-Qubit Bell State Baseline

Let's start with the baseline 2-qubit Bell state $|\Phi^+\rangle$. With ideal simulation, $N = 2$, depth=2, swaps=0, and correlation=1.0, this achieves exactly **1,000 points**.

In [ ]:
```
bell = QuantumCircuit(2, 2, name="Bell State")
bell.h(0)
bell.cx(0, 1)
bell.measure([0, 1], [0, 1])

display(bell.draw('mpl'))
print(f"Quantum gate depth (excluding measurement): {bell.depth() - 1}")
print(f"Operations: {dict(bell.count_ops())}")

# Simulate on AerSimulator
sim = AerSimulator()
counts_bell = sim.run(bell, shots=1000).result().get_counts()
corr_bell = (counts_bell.get('00', 0) + counts_bell.get('11', 0)) /
sum(counts_bell.values())

print(f"Bell state correlation: {corr_bell:.2%}")
display(plot_histogram(counts_bell, title="2-Qubit Bell State
Measurements"))
```

Part 2: Scaling Up — Linear Cascade vs. Parallel Binary Tree

To entangle N qubits, many beginners write a **linear cascade**:

- $H(0)$
- $CX(0, 1)$
- $CX(1, 2)$
- $\dots$
- $CX(N-2, N-1)$

This has depth N. But if we structure the CNOTs as a **binary tree**, we can execute CNOTs in parallel, dropping the entangling depth from $O(N)$ down to $O(\log_2 N)$!

In [ ]:
```python
def build_ghz_linear(n: int) -> QuantumCircuit:
    # Linear cascade GHZ state: depth scales as O(n)
    qc = QuantumCircuit(n, n, name=f"{n}-Qubit Linear GHZ")
    qc.h(0)
    for i in range(n - 1):
        qc.cx(i, i + 1)
    qc.measure(range(n), range(n))
    return qc

def build_ghz_tree(n: int) -> QuantumCircuit:
    # Parallel binary tree GHZ state: depth scales as O(log2(n))
    qc = QuantumCircuit(n, n, name=f"{n}-Qubit Tree GHZ")
    qc.h(0)
    step = 1
    while step < n:
        for i in range(0, n - step, 2 * step):
            qc.cx(i, i + step)
        step *= 2
    qc.measure(range(n), range(n))
    return qc

N = 8
linear_qc = build_ghz_linear(N)
tree_qc = build_ghz_tree(N)

print(f"For N = {N} qubits:")
print(f"  Linear Cascade Depth: {linear_qc.depth()} | Min expected: {1 +
math.ceil(math.log2(N)) + 1}")
print(f"  Parallel Tree Depth:  {tree_qc.depth()} | Min expected: {1 +
math.ceil(math.log2(N)) + 1}")

print("\n--- Parallel Tree Circuit Diagram ---")
display(tree_qc.draw('mpl'))
```

Part 3: Simulating Hardware Realities (Noise, Connectivity & SWAP Gates)

On real hardware, two physical constraints impact your score:

1. **Connectivity Map:** Qubits are connected in a grid or line. If a gate targets two disconnected qubits, Qiskit must insert `swap` gates.
2. **Device Noise:** Every additional gate and unit of depth exposes the qubits to decoherence and depolarizing error.

Let's construct a realistic noisy environment and compare unoptimized transpilation vs. optimized transpilation.

In [ ]:
```
# 1. Setup a linear coupling map (0 - 1 - 2 - 3 - ... - N-1)
line_edges = []
for i in range(N - 1):
    line_edges.extend([(i, i + 1), (i + 1, i)])
line_coupling = CouplingMap(line_edges)

# 2. Setup a hardware-like depolarizing noise model
noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(depolarizing_error(0.001, 1), ["x",
"sx", "rz", "h"])
noise_model.add_all_qubit_quantum_error(depolarizing_error(0.015, 2), ["cx",
"swap"])

noisy_sim = AerSimulator(noise_model=noise_model, seed_simulator=42)

# Transpile tree circuit: unoptimized (level 0) vs optimized (level 3)
transpiled_opt0 = transpile(tree_qc, coupling_map=line_coupling,
optimization_level=0, seed_transpiler=42)
transpiled_opt3 = transpile(tree_qc, coupling_map=line_coupling,
optimization_level=3, seed_transpiler=42)

print("Optimization Level 0:")
print(f"  Depth: {transpiled_opt0.depth()}, Operations:
{dict(transpiled_opt0.count_ops())}")

print("\nOptimization Level 3:")
print(f"  Depth: {transpiled_opt3.depth()}, Operations:
{dict(transpiled_opt3.count_ops())}")
```

Part 4: Measuring and Evaluating Correlation

For an N-qubit GHZ state, the success rate is the fraction of measurement shots where all qubits agree (all-zeros `'0' * N` or all-ones `'1' * N`).

In [ ]:
```
def ghz_correlation_rate(counts: dict, num_qubits: int) → float:
    all_zeros = "0" * num_qubits
    all_ones = "1" * num_qubits
    total_shots = sum(counts.values())
    correlated_shots = counts.get(all_zeros, 0) + counts.get(all_ones, 0)
    return correlated_shots / total_shots

# Run both transpiled circuits on the noisy simulator
counts_opt0 = noisy_sim.run(transpiled_opt0,
shots=1000).result().get_counts()
counts_opt3 = noisy_sim.run(transpiled_opt3,
shots=1000).result().get_counts()

corr_opt0 = ghz_correlation_rate(counts_opt0, N)
corr_opt3 = ghz_correlation_rate(counts_opt3, N)

print(f"Level 0 Correlation: {corr_opt0:.2%}")
print(f"Level 3 Correlation: {corr_opt3:.2%}")

display(plot_histogram([counts_opt0, counts_opt3], legend=["Level 0
(Unoptimized)", "Level 3 (Optimized)"], title=f"{N}-Qubit GHZ Outcomes Under
Noise"))
```

Part 5: Local Score Previewer (Test Before Submitting)

This helper function runs the exact same scoring algorithm as the backend server. Use it to preview your expected score, inspect individual penalties, and tune your circuit!

In [ ]:
```
def preview_challenge_score(circuit: QuantumCircuit, correlation: float):
    # Mirrors the server-side scoreGhzChallenge logic
    num_qubits = circuit.num_qubits
    ops = dict(circuit.count_ops())

    # Calculate depth excluding measurements
    measured_ops = ops.get('measure', 0)
    gate_depth = circuit.depth() - (1 if measured_ops > 0 else 0)
    swap_count = ops.get('swap', 0)

    random_threshold = 1.0 / (2.0 ** (num_qubits - 1))
    base_score = 1000.0 + 50.0 * (num_qubits ** 2 - 4)

    if correlation ⩽ random_threshold:
        print(f"[!] Correlation ({correlation:.2%}) is below or equal to
random noise ({random_threshold:.2%}). Score = 0.")
        return 0

    norm_fidelity = (correlation - random_threshold) / (1.0 -
random_threshold)
    fidelity_factor = norm_fidelity ** 2
    swap_penalty = 1.0 / (1.0 + 0.25 * swap_count)
```

```
    min_theoretical_depth = 1 + math.ceil(math.log2(num_qubits))
    depth_efficiency = min_theoretical_depth / max(gate_depth,
min_theoretical_depth)

    total_score = round(base_score * fidelity_factor * swap_penalty *
depth_efficiency)

    print("=" * 45)
    print(f"SCORE BREAKDOWN ({num_qubits} Qubits)")
    print("=" * 45)
    print(f"  Base Potential:       {base_score:,.0f} pts")
    print(f"  Observed Correlation: {correlation:.2%} (random baseline:
{random_threshold:.2%})")
    print(f"  Fidelity Factor:      {fidelity_factor:.3f}")
    print(f"  SWAP Count:           {swap_count} (penalty:
{swap_penalty:.3f})")
    print(f"  Circuit Gate Depth:   {gate_depth} (min theoretical:
{min_theoretical_depth}, efficiency: {depth_efficiency:.3f})")
    print("-" * 45)
    print(f"  PROJECTED SCORE:      {total_score:,} POINTS")
    print("=" * 45)
    return total_score

print("Preview for Optimization Level 0:")
score_0 = preview_challenge_score(transpiled_opt0, corr_opt0)

print("\nPreview for Optimization Level 3:")
score_3 = preview_challenge_score(transpiled_opt3, corr_opt3)
```

Part 6: Submit to the Hackathon Platform

When you are satisfied with your circuit and score, submit your solution to the live server.

1. Enter your team API key (obtained from your Participant Dashboard).
2. Ensure the submission URL matches your event deployment.
3. Post the QASM export and observed correlation rate.

In [ ]:

```
# Choose which circuit to submit (e.g. your best optimized circuit)
submission_circuit = transpiled_opt3
submission_correlation = corr_opt3

# Configuration
API_KEY = "YOUR_TEAM_API_KEY_HERE"  # Replace with your team API key
SUBMISSION_URL = "http://localhost:3000/api/submissions"
CHALLENGE_ID = "challenge_beginner"  # Beginner challenge

payload = {
    "qasm": qasm2.dumps(submission_circuit),
    "challengeId": CHALLENGE_ID,
    "obtainedResult": float(submission_correlation)
}

print(f"Submitting {submission_circuit.num_qubits}-qubit circuit to
{CHALLENGE_ID}...")
try:
    response = requests.post(
        SUBMISSION_URL,
        json=payload,
        headers={"X-API-Key": API_KEY},
        timeout=20
    )
    if response.ok:
        data = response.json()
        print("\n[✓] Submission successful!")
        print(f"    Submission ID: {data['submission']['id']}")
        print(f"    Official Score: {data['submission']['score']:,} pts")
    else:
        print(f"\n[X] Submission failed ({response.status_code}):")
        print("   ", response.json().get("error", response.text))
except Exception as e:
    print(f"\n[!] Network or request error: {e}")
```